

Rapport de Projet

Construction d'une Chaîne CI/CD

Intégration de SonarQube et Nexus Repository

Projet DevOps : analyse qualité, packaging, publication et gestion des artefacts

Table des matières

1	Introduction	3
2	Architecture générale de la chaîne CI/CD	3
3	Préparation du projet	3
3.1	Clonage du dépôt Git	3
3.2	Installation et vérification de Maven	4
4	Mise en place de SonarQube	4
4.1	Rôle de SonarQube	4
4.2	Lancement de SonarQube avec Docker	5
4.3	Configuration Maven pour l'analyse SonarQube	5
4.4	Couverture de tests avec JaCoCo	6
5	Analyse du frontend Angular avec SonarQube	6
5.1	Création du projet local	6
5.2	Résultats d'analyse	6
6	Analyse du backend Spring Boot avec SonarQube	7
6.1	Création et configuration du projet backend	7
6.2	Génération du token d'accès	8
6.3	Choix de la méthode d'analyse	8
6.4	Amélioration après ajout des tests	8
7	Mise en place de Nexus Repository	9
7.1	Rôle de Nexus	9
7.2	Packaging du backend Spring Boot	9
7.3	Packaging du frontend Angular	10

7.4	Authentification et accès à Nexus	10
8	Configuration des dépôts Nexus	10
8.1	Repository backend	10
8.2	Repository proxy	10
8.3	Repository group	11
8.4	Repository frontend	11
9	Problème rencontré et solution appliquée	11
	Deuxième partie : Intégration automatique avec Jenkins	16
	Résultat du build Jenkins	24
	Conclusion	25
	Références utiles	25

1 Introduction

Ce rapport présente la construction d'une chaîne CI/CD sécurisée pour un projet composé d'un backend Spring Boot et d'un frontend Angular. L'objectif principal est de mettre en place un processus fiable permettant de récupérer le code source, de compiler les applications, d'analyser leur qualité avec SonarQube, puis de publier les livrables dans Nexus Repository.

La démarche suivie permet de renforcer la qualité logicielle avant livraison. SonarQube est utilisé pour contrôler la maintenabilité, les bugs, les vulnérabilités, les duplications et la couverture des tests. Nexus est utilisé comme dépôt centralisé pour stocker les artefacts générés, notamment les fichiers `.jar` du backend et les archives `.zip` du frontend.

Objectifs du projet

- Cloner et préparer le projet applicatif depuis Git.
- Installer et vérifier l'environnement Maven nécessaire au backend Spring Boot.
- Déployer SonarQube avec Docker et configurer les projets d'analyse.
- Exécuter l'analyse qualité pour le frontend Angular et le backend Spring Boot.
- Générer les livrables applicatifs et les publier dans Nexus Repository.
- Organiser les dépôts Nexus selon les besoins : *hosted*, *proxy* et *group*.

2 Architecture générale de la chaîne CI/CD

La chaîne réalisée repose sur une organisation classique DevOps. Le code source est récupéré depuis un dépôt Git, les dépendances sont résolues avec Maven ou npm selon la partie du projet, les analyses sont envoyées vers SonarQube et les artefacts finaux sont publiés dans Nexus.

Composant	Rôle dans la chaîne CI/CD
Git	Centralise le code source et permet de cloner le projet localement.
Maven	Compile le backend Spring Boot, lance les tests et génère le fichier <code>.jar</code> .
npm/Angular	Installe les dépendances frontend et produit les fichiers de distribution.
SonarQube	Analyse la qualité du code, la maintenabilité, les bugs et la sécurité.
Docker	Fournit un environnement simple et reproductible pour exécuter SonarQube.
Nexus	Stocke et organise les artefacts backend et frontend.

3 Préparation du projet

3.1 Clonage du dépôt Git

La première étape consiste à récupérer le code source depuis Git. Cette opération permet de travailler sur une copie locale du projet et de préparer les phases de build, de test et d'analyse.

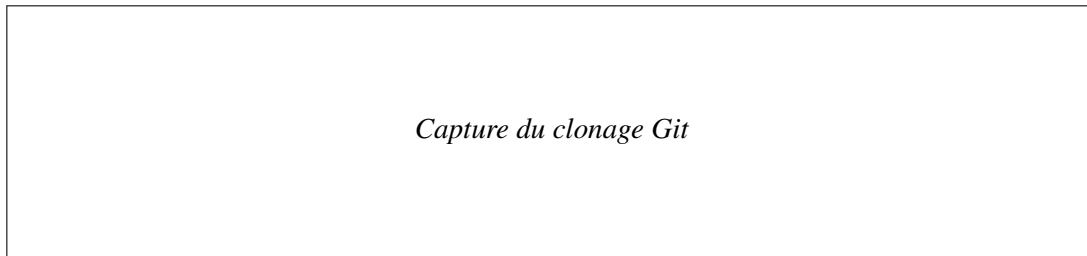


FIGURE 1 – Clonage du projet depuis le dépôt Git.

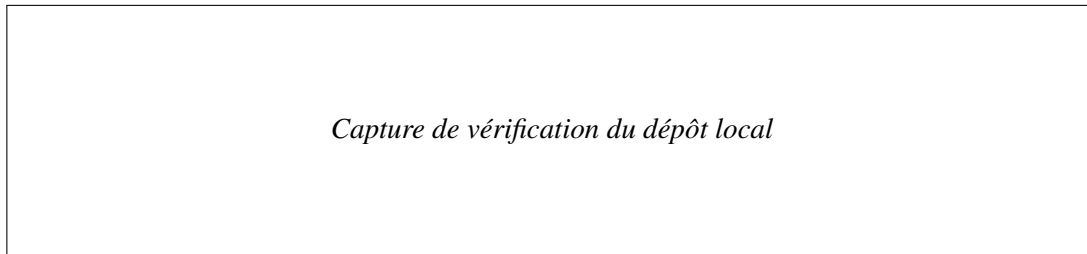


FIGURE 2 – Vérification de la récupération du projet en local.

3.2 Installation et vérification de Maven

Maven est indispensable pour la partie backend Spring Boot. Il permet de gérer les dépendances, de compiler le projet, d'exécuter les tests unitaires et de générer le livrable final. La vérification de l'installation confirme que l'environnement est prêt pour l'analyse et le packaging.

4 Mise en place de SonarQube

4.1 Rôle de SonarQube

SonarQube est une plateforme d'inspection continue de la qualité du code. Elle permet de détecter les erreurs, les vulnérabilités, les mauvaises pratiques, les duplications et les problèmes de maintenabilité. Dans cette chaîne CI/CD, SonarQube intervient avant la publication des livrables afin de garantir que le code respecte un niveau de qualité acceptable.

Le serveur SonarQube démarre principalement trois processus :

- **Web Server** : fournit l'interface web utilisée par les administrateurs et les développeurs.
- **Search Server** : repose sur Elasticsearch et permet d'effectuer les recherches dans l'interface.
- **Compute Engine Server** : traite les rapports d'analyse envoyés par les scanners et enregistre les résultats dans la base de données SonarQube.

La base de données SonarQube conserve la configuration de l'instance, les paramètres des projets, l'historique des analyses et les rapports de qualité.

4.2 Lancement de SonarQube avec Docker

SonarQube a été lancé via Docker sur le port 9000. Cette approche simplifie le déploiement, car elle évite une installation manuelle lourde et permet d'obtenir rapidement une instance exploitable.

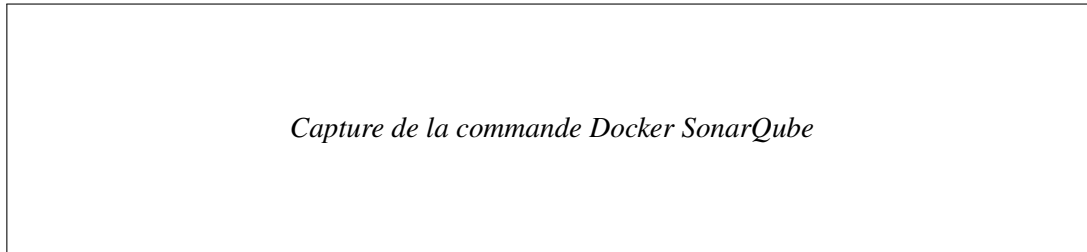


FIGURE 3 – Commande de lancement de SonarQube avec Docker sur le port 9000.

Après le démarrage, l'interface est accessible depuis le navigateur à l'adresse `http://localhost:9000`. Les identifiants initiaux utilisés sont `admin/admin`, puis un nouveau mot de passe est configuré lors de la première connexion.

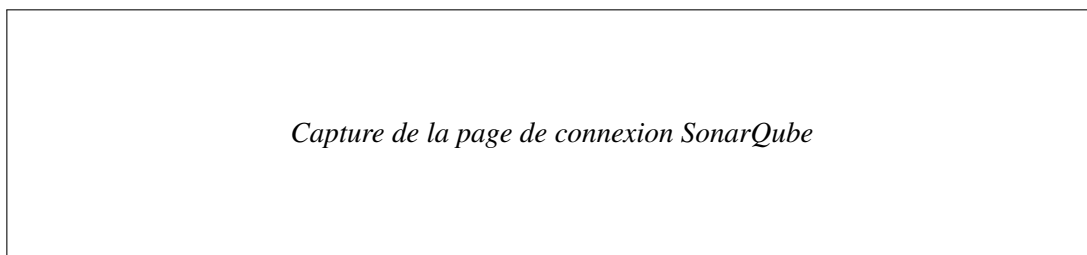


FIGURE 4 – Page de connexion SonarQube accessible sur le port 9000.

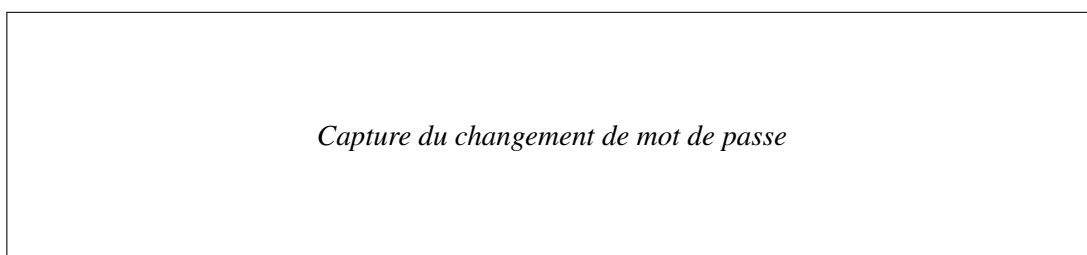


FIGURE 5 – Changement du mot de passe administrateur après la première connexion.

4.3 Configuration Maven pour l'analyse SonarQube

Pour permettre l'analyse du backend Spring Boot, le plugin SonarQube est ajouté dans le fichier `pom.xml`, au niveau de la section `build/plugins`. Cette configuration permet à Maven d'envoyer les résultats d'analyse vers le serveur SonarQube.

4.4 Couverture de tests avec JaCoCo

JaCoCo est ajouté à la partie backend afin de mesurer la couverture des tests unitaires. Il génère un rapport de couverture qui peut être lu par SonarQube pendant l'analyse Maven. Cette intégration permet de visualiser dans SonarQube le pourcentage de code testé, les lignes non couvertes et les zones du backend qui nécessitent davantage de tests.

Dans une chaîne CI/CD, JaCoCo complète donc l'analyse SonarQube : SonarQube évalue la qualité globale du code, tandis que JaCoCo fournit les métriques de couverture nécessaires pour valider la fiabilité des tests avant la livraison.

5 Analyse du frontend Angular avec SonarQube

5.1 Création du projet local

Dans SonarQube, un projet local est créé depuis le menu *Projects*, puis *Create a local project*. Cette étape déclare le projet dans la plateforme afin de pouvoir envoyer les résultats d'analyse du frontend Angular.

Une fois le projet créé, SonarQube propose une méthode d'analyse. Le scanner est utilisé localement afin de lancer l'analyse depuis le dossier du projet Angular.

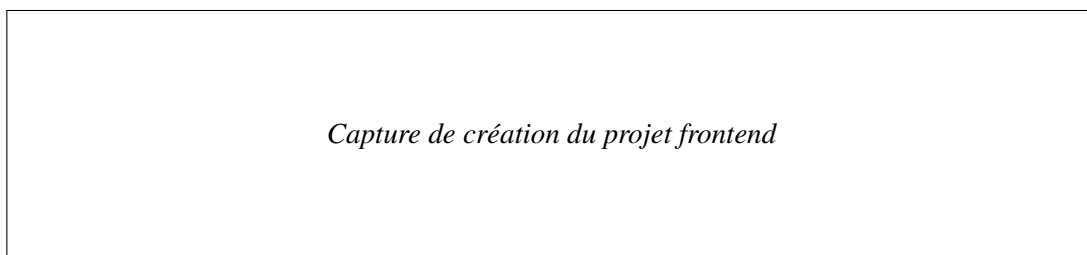


FIGURE 6 – Création du projet frontend Angular dans SonarQube.

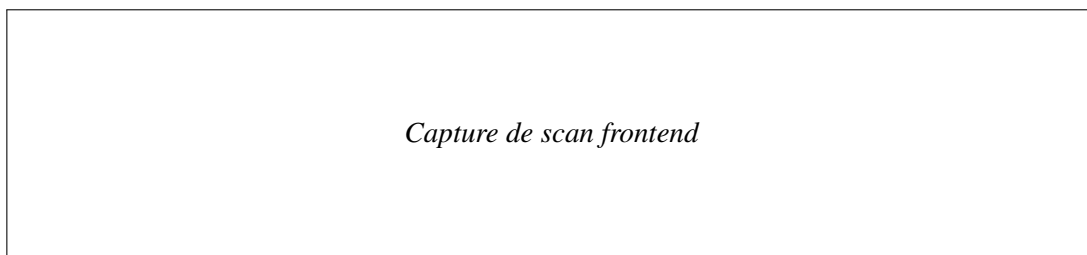


FIGURE 7 – Exécution de la commande d'analyse SonarQube pour le frontend Angular.

5.2 Résultats d'analyse

Après exécution de l'analyse, SonarQube affiche un tableau de bord permettant de suivre l'état global du projet. Les indicateurs visibles concernent notamment la qualité globale, la maintenabilité, la fiabilité, la sécurité et les duplications.

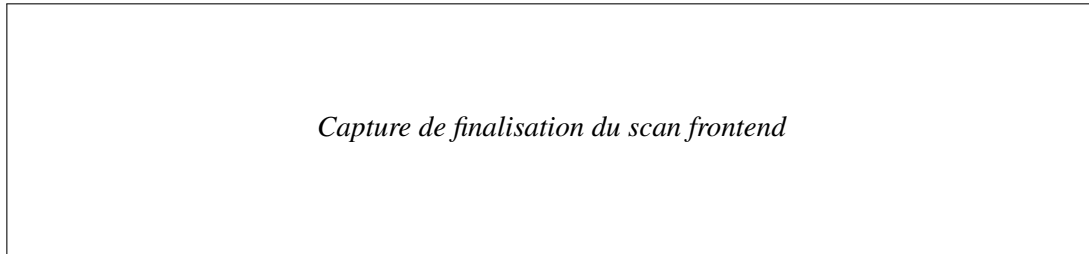


FIGURE 8 – Finalisation de l’analyse du projet frontend dans SonarQube.

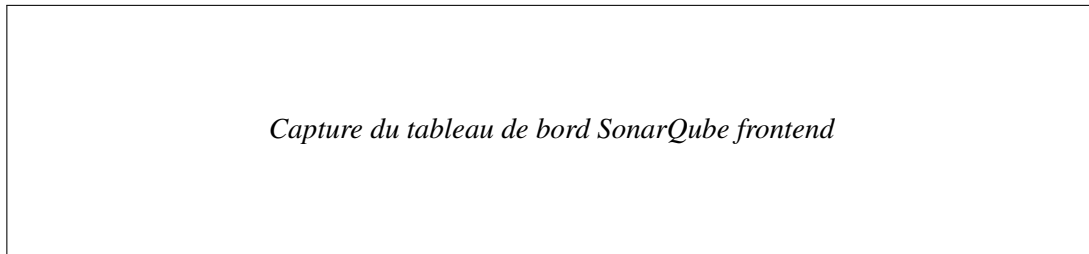


FIGURE 9 – Résultat global de l’analyse SonarQube du frontend.

Ces résultats permettent d’identifier les points à corriger avant intégration ou livraison. La maintenabilité est particulièrement importante, car elle mesure l’effort nécessaire pour comprendre, modifier et faire évoluer le code.

6 Analyse du backend Spring Boot avec SonarQube

6.1 Création et configuration du projet backend

Un second projet local est créé pour le backend Spring Boot. La séparation frontend/backend permet d’obtenir des métriques indépendantes pour chaque partie de l’application et de suivre plus précisément la qualité de chaque composant.

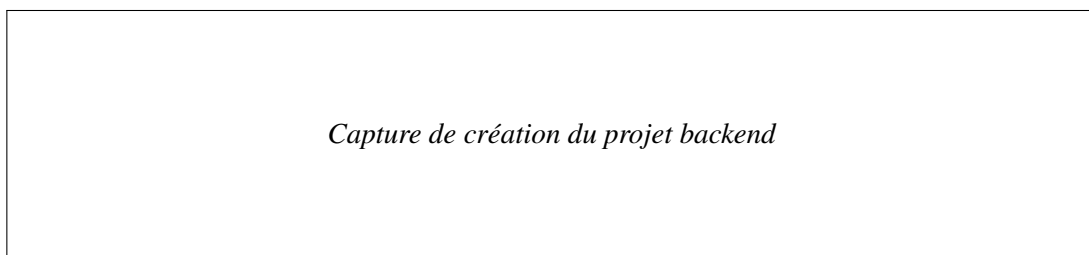


FIGURE 10 – Création du projet backend dans SonarQube.

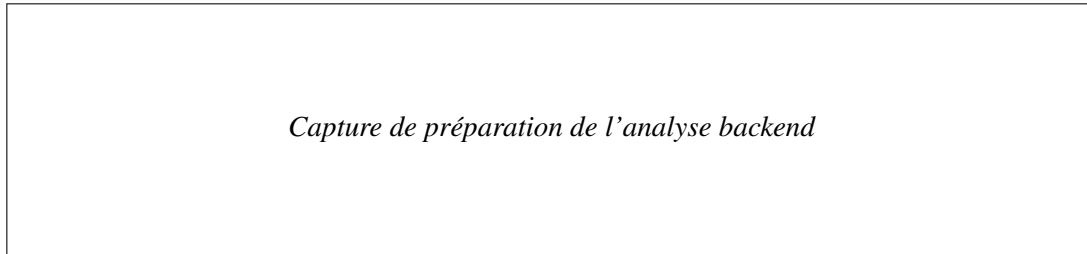


FIGURE 11 – Écran de préparation de l'analyse backend.

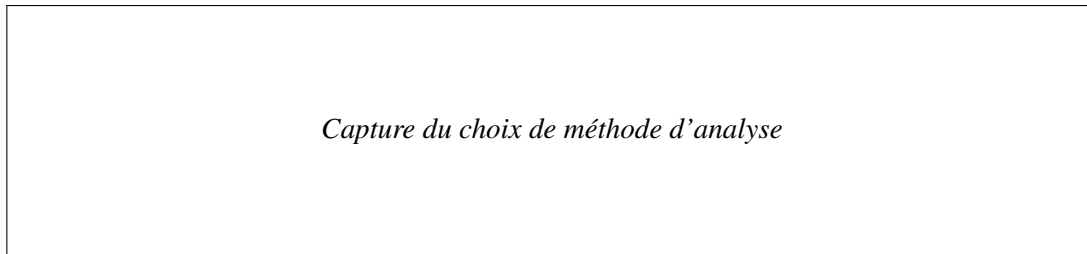


FIGURE 12 – Choix de la méthode d'analyse du backend.

6.2 Génération du token d'accès

SonarQube nécessite un token pour authentifier les analyses lancées depuis la machine locale ou depuis un pipeline CI/CD. Le token est généré depuis *My Account*, puis *Security*, dans la section *Tokens*. Une durée de validité peut être définie selon la politique de sécurité choisie.

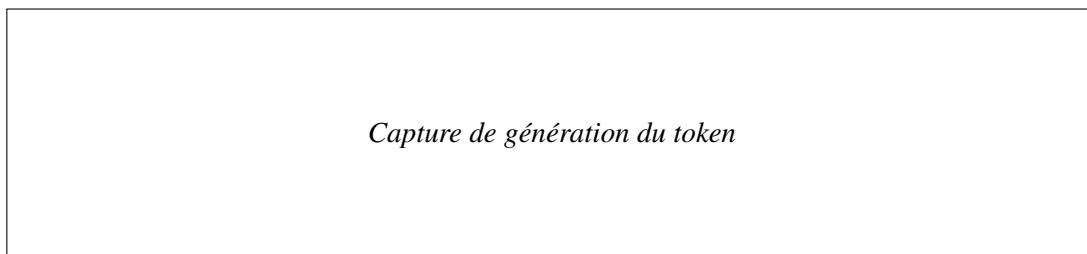


FIGURE 13 – Génération d'un token d'accès dans SonarQube.

6.3 Choix de la méthode d'analyse

La méthode d'analyse sélectionnée est *Locally*. Dans l'étape suivante, l'option *Use existing token* est choisie afin d'utiliser le token généré précédemment. Maven est ensuite sélectionné comme outil de build pour lancer l'analyse du backend.

6.4 Amélioration après ajout des tests

Après l'ajout des tests, une nouvelle analyse est exécutée. Cette étape permet de mesurer l'impact des tests sur la qualité du projet, notamment sur la fiabilité et la couverture. L'intégration de tests dans la chaîne

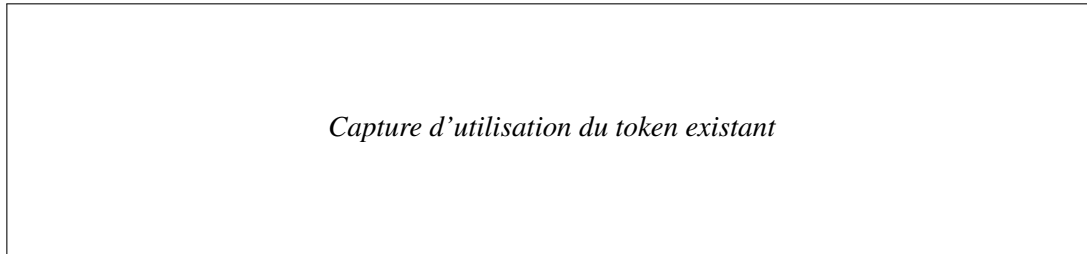


FIGURE 14 – Utilisation d'un token existant pour l'analyse locale.

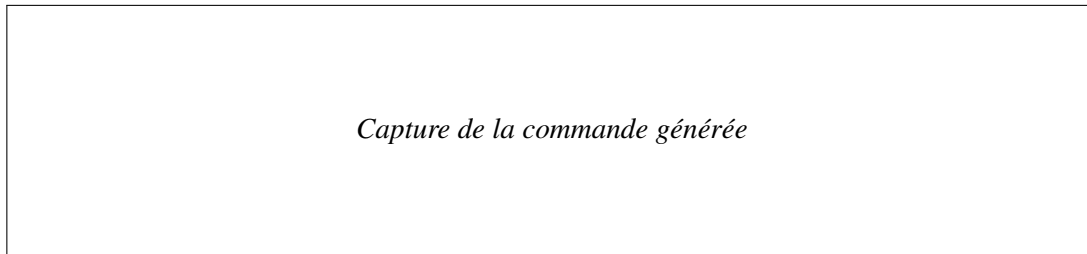


FIGURE 15 – Commande générée par SonarQube pour le scan backend.

CI/CD améliore la confiance avant publication du livrable.

7 Mise en place de Nexus Repository

7.1 Rôle de Nexus

Nexus Repository est utilisé comme gestionnaire d'artefacts. Il centralise les livrables générés par les builds et facilite leur réutilisation dans les environnements de développement, de test, de préproduction ou de production.

Dans ce projet, Nexus est utilisé pour :

- stocker le livrable backend Spring Boot au format `.jar` ;
- stocker le livrable frontend Angular sous forme d'archive `.zip` ;
- organiser les dépôts selon leur rôle : *hosted*, *proxy* et *group* ;
- sécuriser l'accès aux artefacts par authentification.

7.2 Packaging du backend Spring Boot

Dans le projet Spring Boot, le build Maven permet de générer le fichier `.jar`. Ce fichier représente le livrable backend qui sera exécuté puis publié dans Nexus.

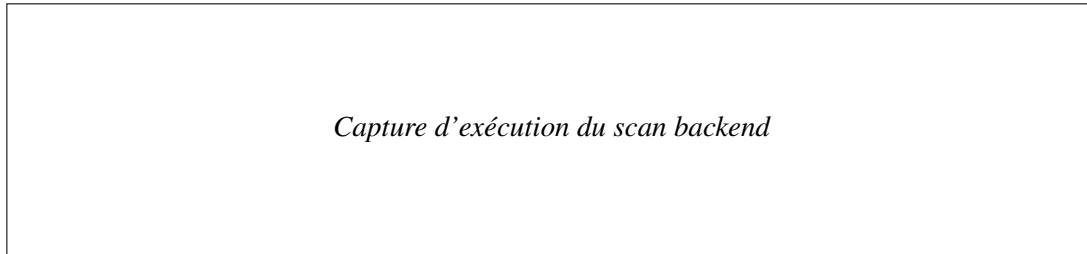


FIGURE 16 – Exécution de la commande d’analyse SonarQube pour le backend Spring Boot.

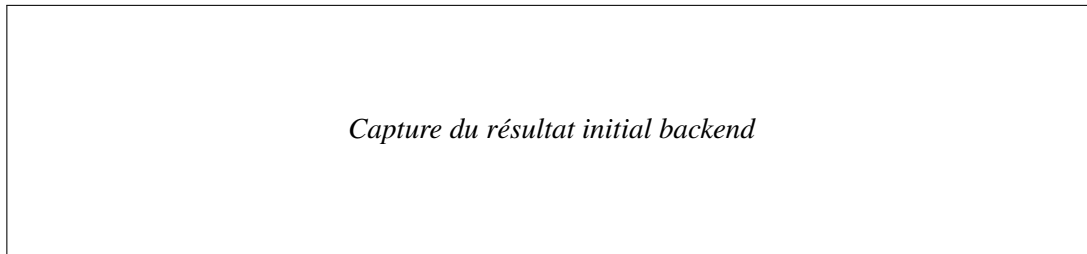


FIGURE 17 – Résultat du scan backend avant l’augmentation de la couverture de tests.

7.3 Packaging du frontend Angular

Pour la partie Angular, les dépendances sont installées et le projet est construit afin de générer les fichiers statiques de distribution. Ces fichiers sont ensuite compressés dans une archive .zip, ce qui facilite leur publication dans Nexus.

7.4 Authentification et accès à Nexus

L'accès à Nexus est protégé par un compte administrateur. Après connexion, il est possible de créer et d'administrer les dépôts nécessaires au projet.

8 Configuration des dépôts Nexus

8.1 Repository backend

Un dépôt dédié au backend est créé afin d'héberger les livrables Spring Boot. Le type de dépôt utilisé pour publier un artefact interne est généralement *hosted*. Ce dépôt devient la source officielle des versions backend générées par la chaîne CI/CD.

8.2 Repository proxy

Un dépôt *proxy* permet à Nexus de servir d'intermédiaire vers un dépôt distant. Il est utile pour mettre en cache les dépendances externes et améliorer la stabilité des builds, même lorsque le réseau ou le dépôt

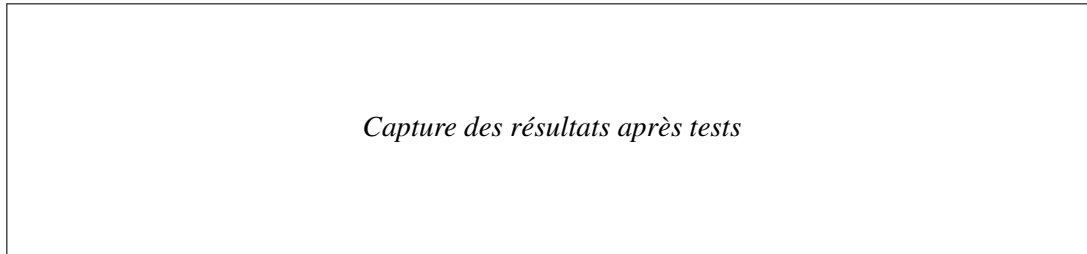


FIGURE 18 – Résultats après ajout et exécution des tests.

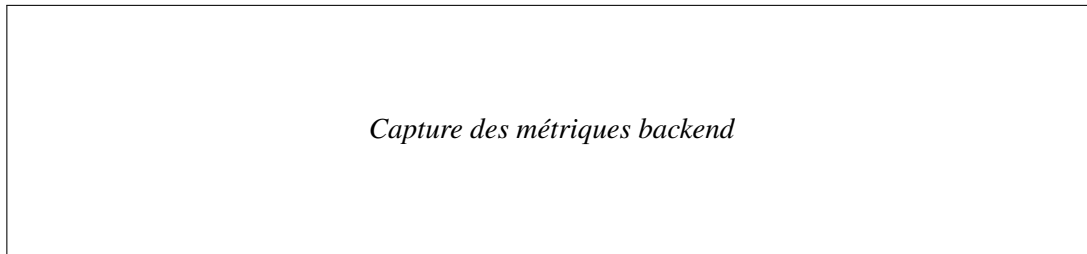


FIGURE 19 – Synthèse des métriques backend après analyse.

distant est temporairement indisponible.

8.3 Repository group

Un dépôt *group* permet de regrouper plusieurs repositories sous une seule URL. Cette approche simplifie la configuration des clients, car Maven ou npm peut utiliser une URL unique pour accéder à plusieurs sources d'artefacts.

8.4 Repository frontend

Pour le frontend, un repository *raw hosted* est créé. Ce type de dépôt est adapté au stockage de fichiers génériques comme des archives .zip. L'archive générée à partir du build Angular est ensuite téléversée dans ce dépôt.

9 Problème rencontré et solution appliquée

Lors du lancement de SonarQube, un problème peut apparaître à cause d'Elasticsearch. SonarQube utilise Elasticsearch en interne, et celui-ci peut échouer si le paramètre système `vm.max_map_count` est trop bas.

La solution consiste à augmenter cette valeur au niveau du système hôte avant de relancer le conteneur SonarQube. Ce réglage est important dans un environnement Linux, car Elasticsearch exige un nombre suffisant de zones mémoire mappées pour fonctionner correctement.

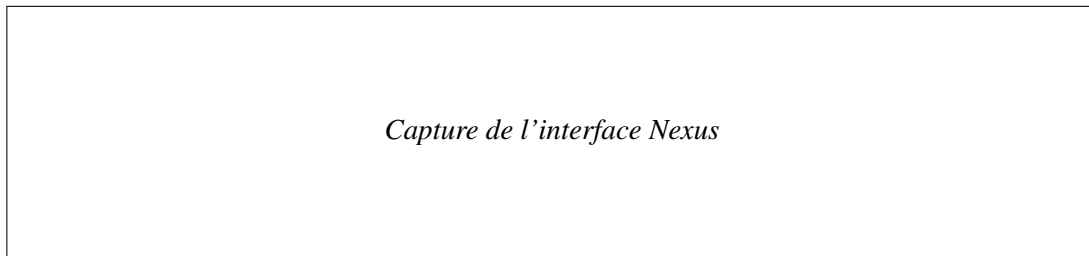


FIGURE 20 – Interface Nexus Repository utilisée pour la gestion des artefacts.



FIGURE 21 – Exécution du build Maven dans le projet Spring Boot.

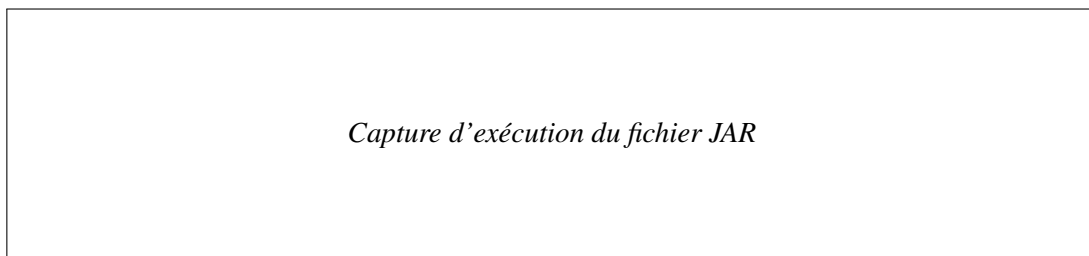


FIGURE 22 – Exécution du fichier JAR généré pour le backend.

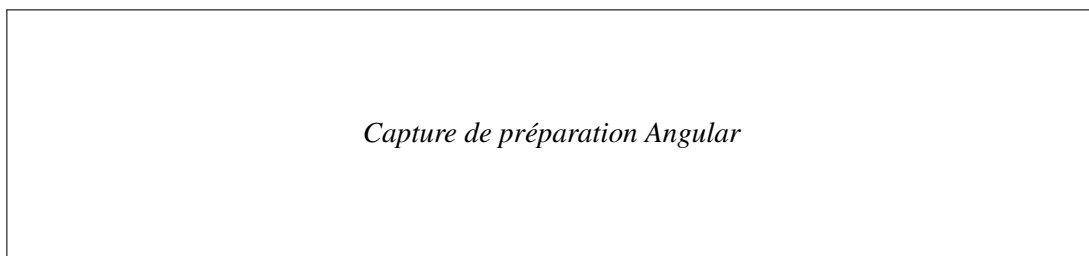


FIGURE 23 – Préparation du projet Angular avant génération du livrable.

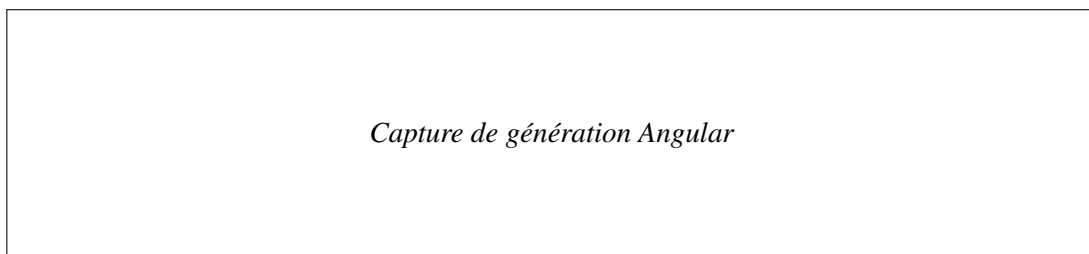


FIGURE 24 – Génération des fichiers de distribution Angular.

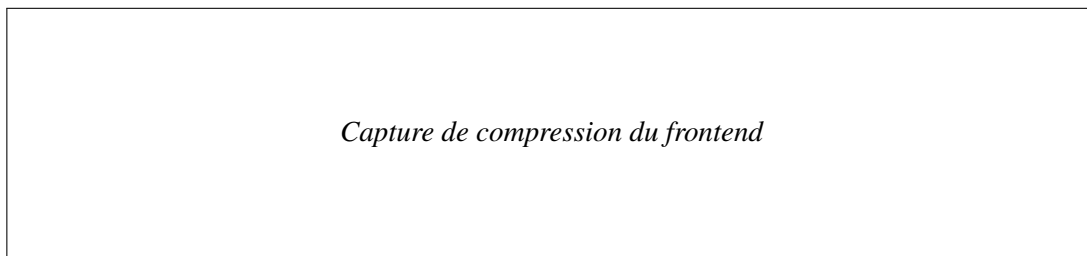


FIGURE 25 – Compression du livrable frontend.

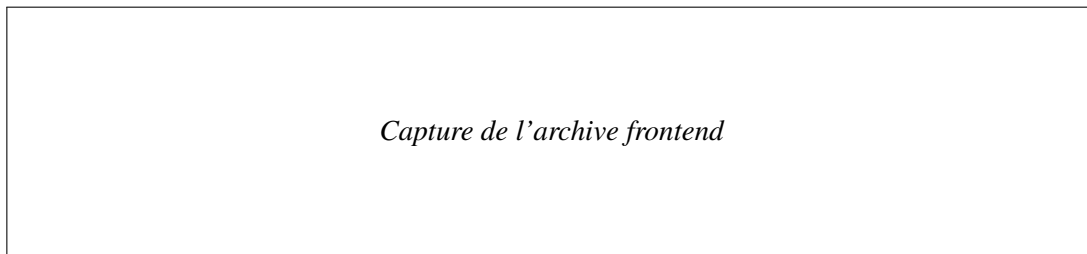


FIGURE 26 – Archive frontend prête à être publiée.

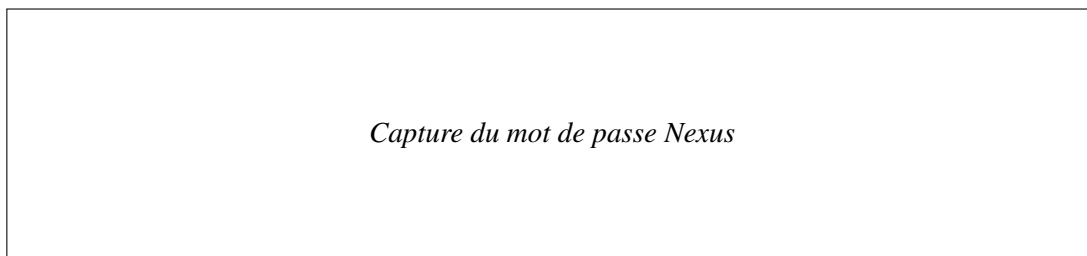


FIGURE 27 – Commande ou information liée au mot de passe Nexus.

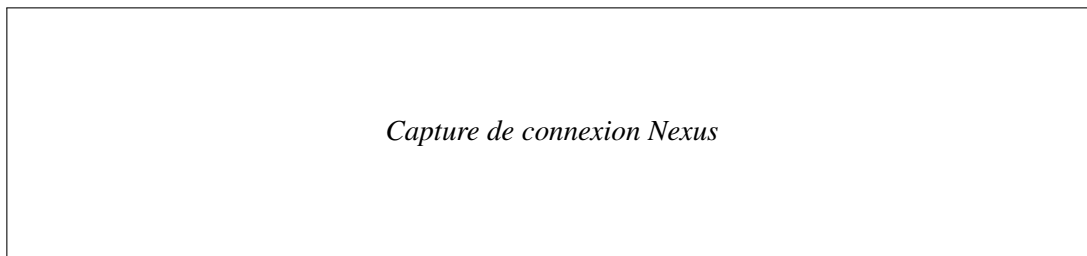


FIGURE 28 – Page de connexion à Nexus Repository.

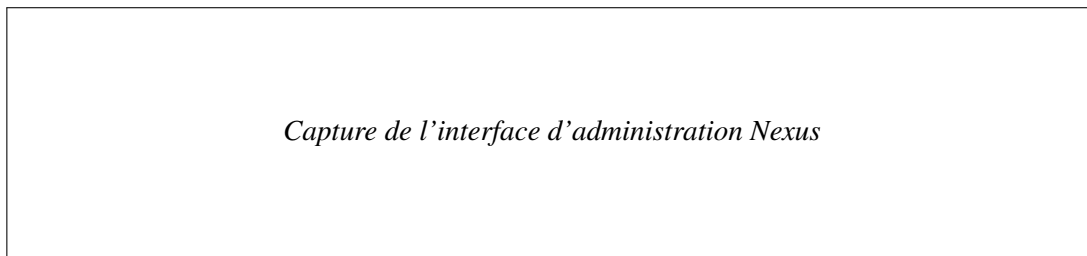


FIGURE 29 – Interface d'administration après authentification.

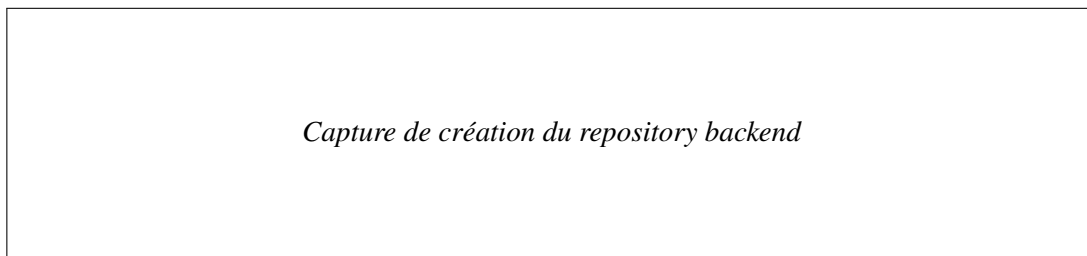


FIGURE 30 – Création ou configuration du repository backend.

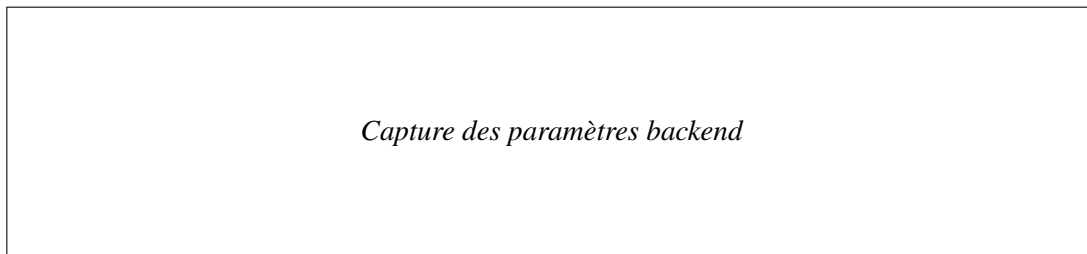


FIGURE 31 – Paramètres du dépôt backend.

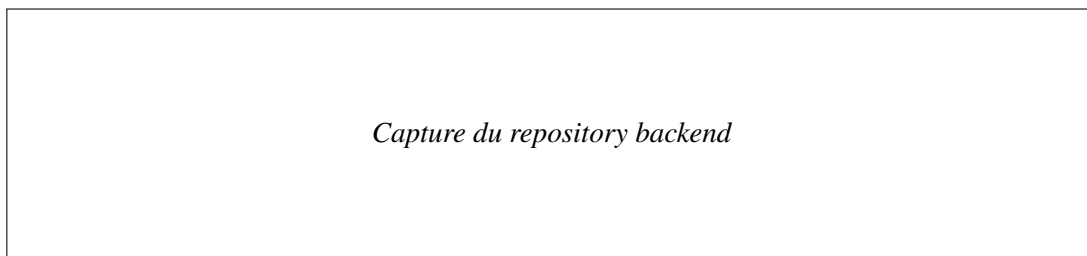


FIGURE 32 – Dépôt backend disponible dans Nexus.

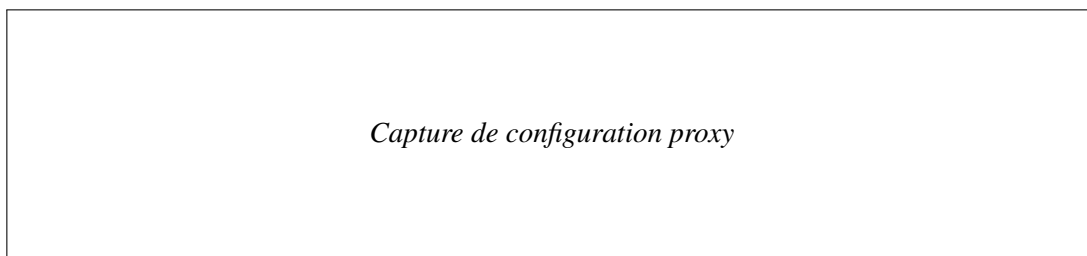


FIGURE 33 – Configuration d'un repository de type proxy.

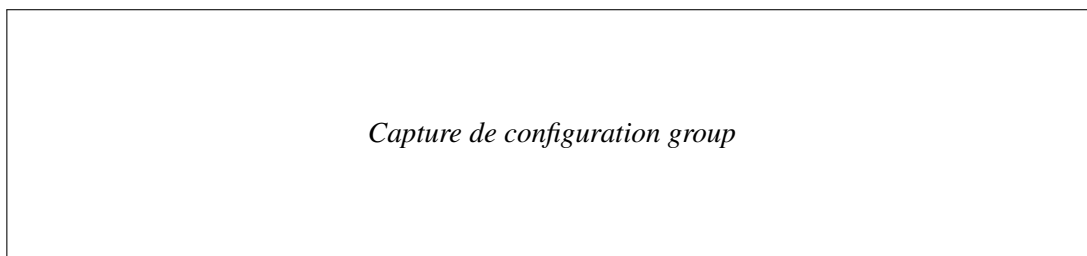
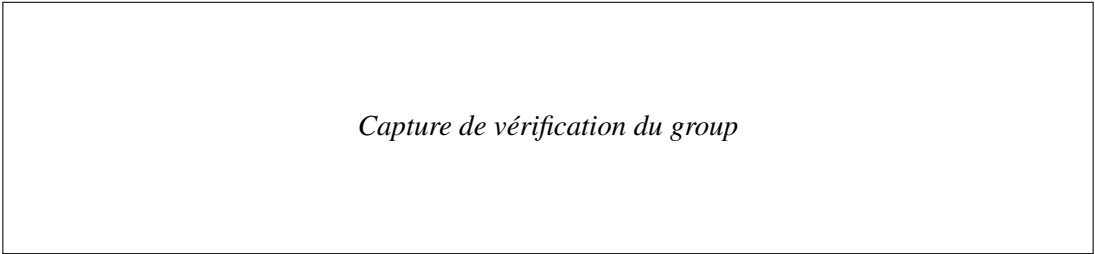


FIGURE 34 – Configuration d'un repository de type group.



Capture de vérification du group

FIGURE 35 – Vérification du repository group créé.



Capture de la liste des repositories

FIGURE 36 – Liste des repositories configurés dans Nexus.



Capture de l'URL Nexus

FIGURE 37 – Commande ou URL utilisée pour accéder aux artefacts Nexus.



Capture du repository frontend

FIGURE 38 – Création du repository raw hosted pour le frontend.



Capture de l'erreur Elasticsearch

FIGURE 39 – Erreur Elasticsearch liée au paramètre système `vm.max_map_count`.

Deuxième partie : Intégration automatique avec Jenkins

ÉTAPE 1 — Préparer SonarQube pour Jenkins

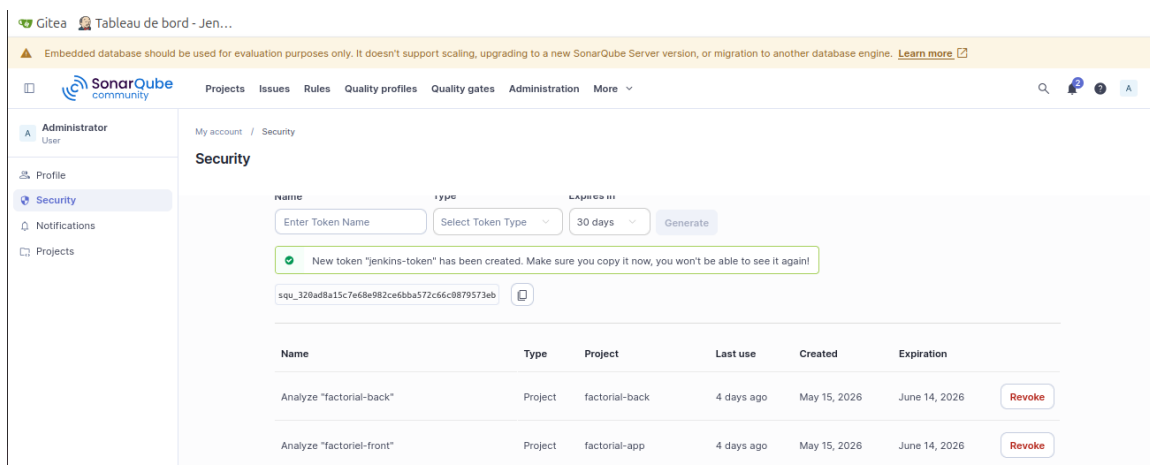
SonarQube → My Account → Security

→ Generate Token

Name : jenkins-token

Type : User Token

→ Copie : squ



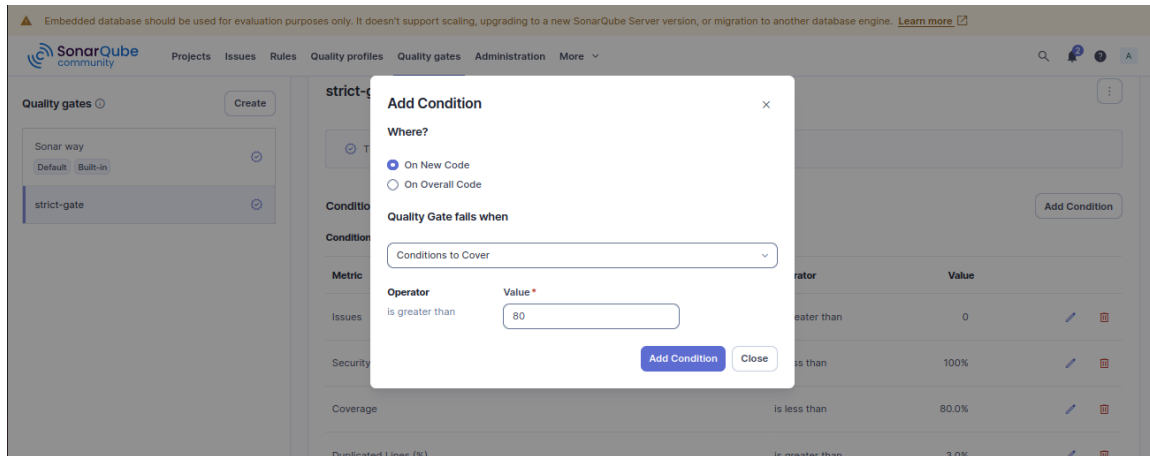
Créer un Quality Gate qui fait échouer le pipeline

SonarQube → Quality Gates → Create

Name : strict-gate

Add Condition :

Coverage is less than 80% ← pipeline échoue si pas de tests



ÉTAPE 2 — Préparer Nexus pour Jenkins

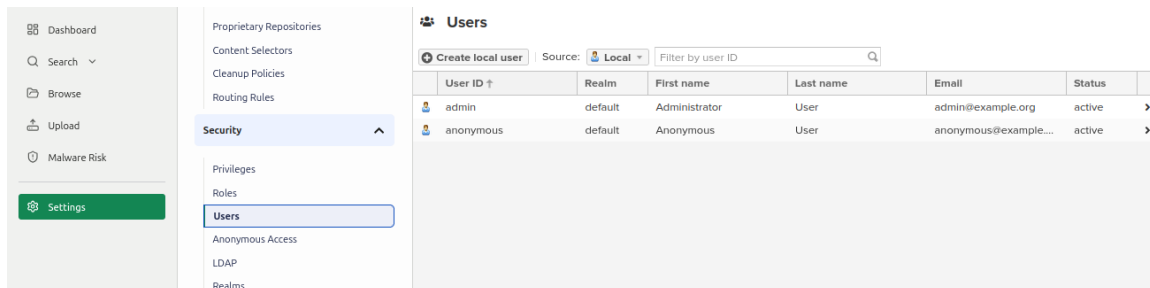
Créer l'utilisateur Jenkins dans Nexus

Nexus → Security → Users → Create

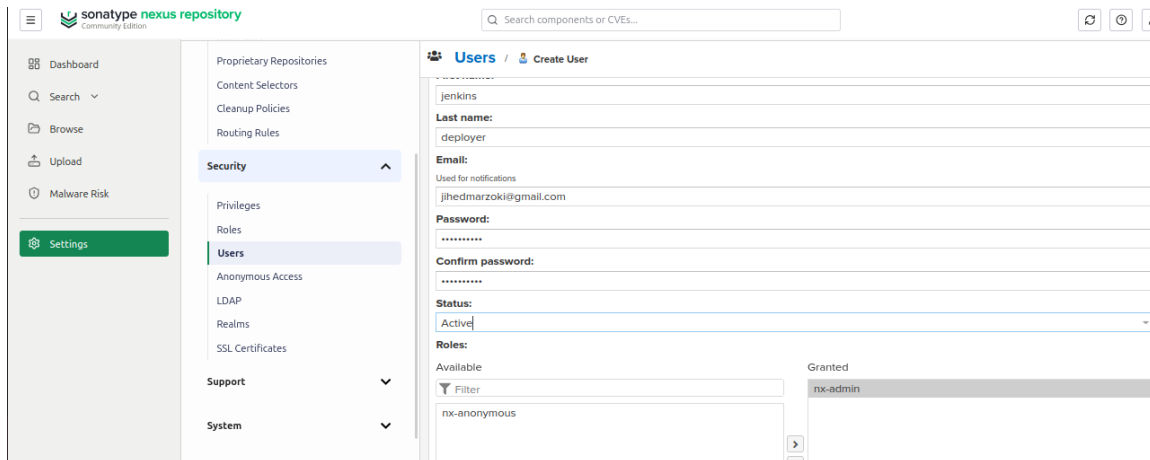
ID : jenkins-deployer

Password : jenkins123

Roles : nx-admin



Crée un user



ÉTAPE 3 — Configurer Jenkins

3.1 Installer les plugins nécessaires

Jenkins → Manage Jenkins → Plugins → Available

Installe :

SonarQube Scanner

Nexus Artifact Uploader

Pipeline

Git

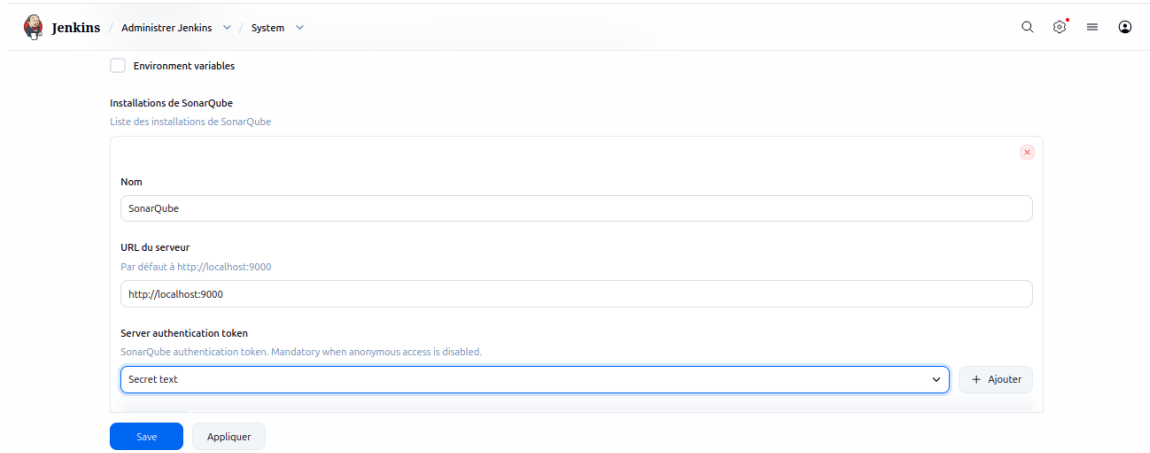


Jenkins → Manage Jenkins → System → SonarQube servers → Add

Name : SonarQube

URL : http ://localhost :9000

Token : colle le token



Jenkins / Administrer Jenkins / System

☐ Environment variables

Installations de SonarQube

Liste des installations de SonarQube

Nom

SonarQube

URL du serveur

Par défaut à http://localhost:9000

http://localhost:9000

Server authentication token

SonarQube authentication token. Mandatory when anonymous access is disabled.

Secret text

+ Ajouter

Save Appliquer

Connecter Jenkins → Nexus

Jenkins → Manage Jenkins → Credentials → Add

Kind : Username with password

Username : admin

Password : jenkins123

ID : nexus-credentials

<

Add Nom d'utilisateur et mot de passe

×

Portée ?
Global (Jenkins, agents, items, etc...) ▼

Nom d'utilisateur
jenkins-deployer

☐ Treat username as secret ?

Mot de passe
.....

ID ?
nexus-credentials

Description ?

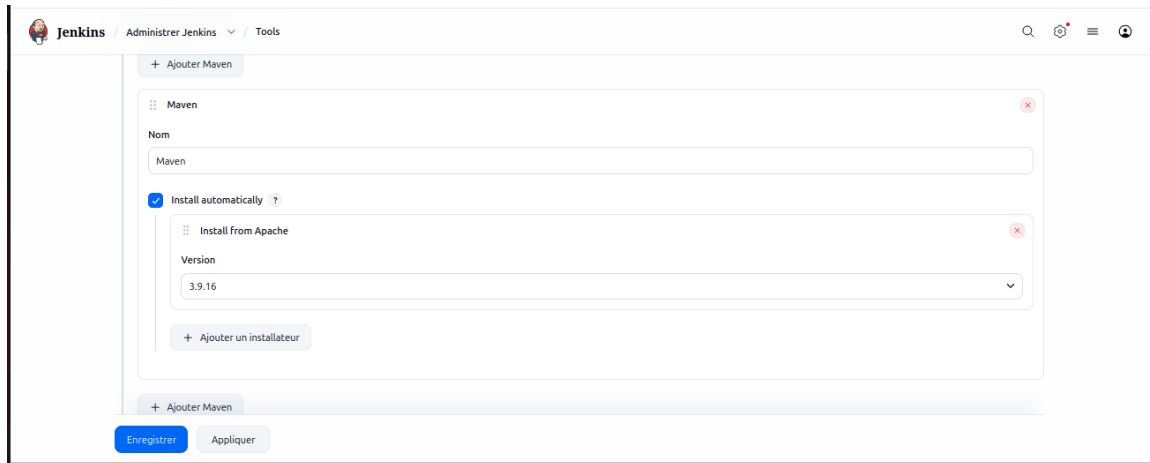
Create

Configurer Maven dans Jenkins

Jenkins → Manage Jenkins → Tools → Maven

Name : Maven

Version : 3.9.6



Ajouter distributionManagement dans pom.xml

```
<distributionManagement>
  <repository>
    <id>nexus-releases</id>
    <url>http://localhost:8081/repository/maven-releases/</url>
  </repository>
  <snapshotRepository>
    <id>nexus-snapshots</id>
    <url>http://localhost:8081/repository/maven-snapshots/</url>
  </snapshotRepository>
</distributionManagement>
```

Ajouter les credentials Nexus dans settings.xml

```
nano ~/.m2/settings.xml
```

```
<settings>
  <servers>
    <server>
      <id>nexus-releases</id>
      <username>jenkins-deployer</username>
      <password>jenkins123</password>
    </server>
    <server>
      <id>nexus-snapshots</id>
      <username>jenkins-deployer</username>
      <password>jenkins123</password>
    </server>
  </servers>
</settings>
```

Créer le Jenkinsfile

Dans le dossier de ton projet Spring Boot :

nano ~/Bureau/ci-cd/project-sonar-nexus/Jenkinsfile

```
useradm@devops:~/Bureau/ci-cd/project-sonar-nexus$ cat Jenkinsfile
pipeline {
  agent any

  tools {
    maven 'Maven'
  }

  stages {
    stage('Checkout') {
      steps {
        echo 'Récupération du code...'
        checkout scm
      }
    }

    stage('Build') {
      steps {
        echo 'Compilation...'
        sh 'mvn clean package -DskipTests'
      }
    }
  }
}
```

Github user



Créer le job Jenkins

Jenkins → New Item

Name : factorial-pipeline

Type : Pipeline

→ Pipeline section :

Definition : Pipeline script from SCM

SCM : Git

URL : <https://github.com/MarzoukiJihed/project-sonar-nexus.git>

Branch : main

Puis builder le pipeline :



Résultat du build Jenkins



factorial-pipeline

SonarQube Quality Gate

factorial-back **Passed**

server-side processing: **Success**

Liens permanents

- [Dernier build \(#16\), il y a 13 mn](#)
- [Dernier build stable \(#16\), il y a 13 mn](#)
- [Dernier build avec succès \(#16\), il y a 13 mn](#)
- [Dernier build non réussi \(#15\), il y a 14 mn](#)
- [Dernier build complété \(#16\), il y a 13 mn](#)

11 Conclusion

Ce projet démontre la mise en place d'une chaîne CI/CD complète autour d'un projet Spring Boot et Angular. L'utilisation de SonarQube améliore la qualité du code grâce à une analyse continue, tandis que Nexus garantit une gestion centralisée et structurée des livrables.

La solution obtenue constitue une base solide pour une industrialisation plus avancée. Elle peut être enrichie par l'intégration d'un outil d'orchestration comme Jenkins, GitLab CI ou GitHub Actions afin d'automatiser entièrement les étapes de build, test, analyse SonarQube et publication dans Nexus.

Références utiles

- Documentation Sonatype sur les types de repositories : <https://help.sonatype.com/en/repository-types.html>
- Documentation Sonatype sur les raw repositories : <https://help.sonatype.com/en/raw-repositories.html>
- Documentation complémentaire sur Nexus, registre Docker privé et Docker Hub proxy : <https://mossaabfrifita.github.io/docs/devops/Cr%C3%A9ez%20un%20registre%20docker%20priv%C3%A9%20et%20un%20docker%20hub%20proxy%20avec%20Nexus>
- Discussion et support de rédaction utilisés pour l'organisation du rapport : <https://chat.deepseek.com/a/chat/s/04cfd72f-e03c-4c64-b721-573975f2e229>
- Interface SonarQube locale utilisée dans le projet : <http://localhost:9000>